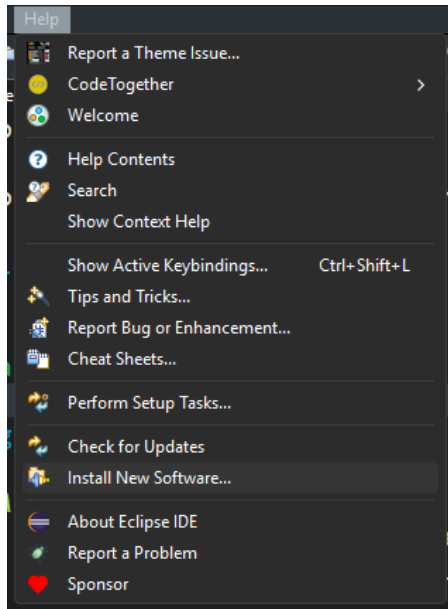


Ejercicio 2 – EcEmma para cobertura de tests

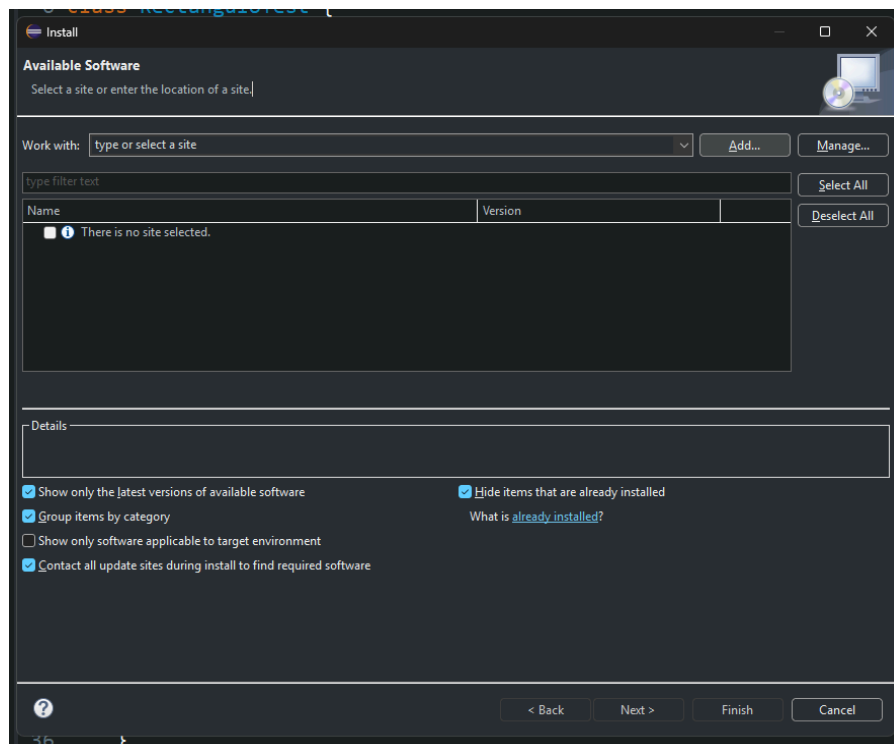
Instalación de EcEmma

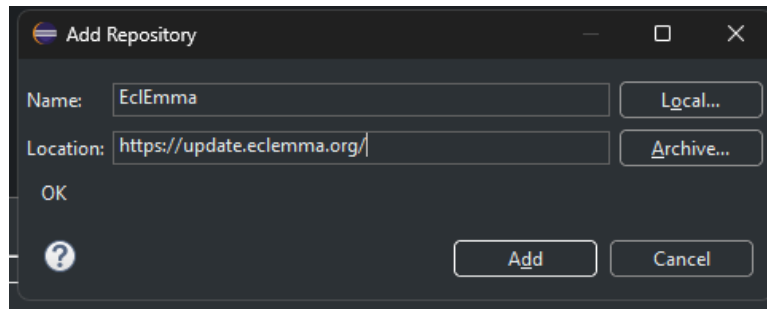
Para realizar el análisis de cobertura de las pruebas unitarias se ha utilizado el plugin EcEmma para Eclipse.



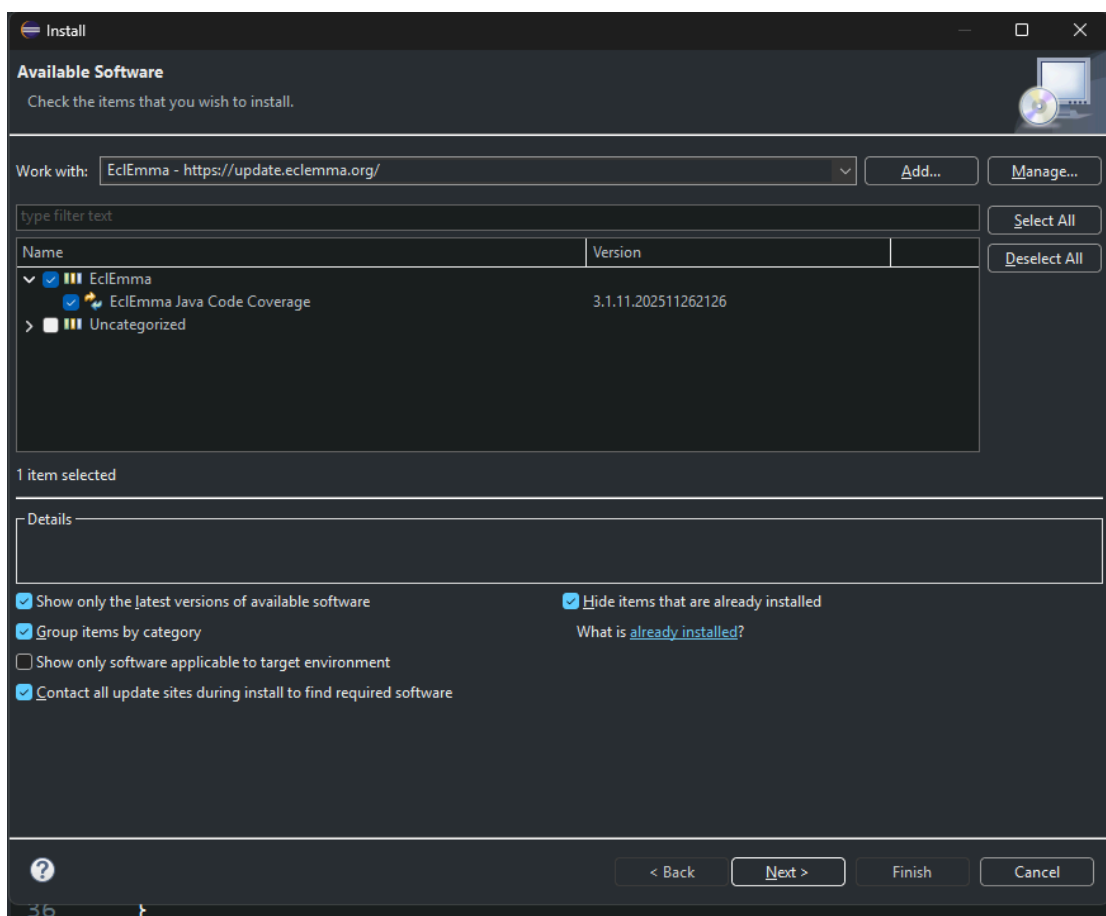
En primer lugar, se accedió al menú **Help** → **Install New Software**, desde donde es posible instalar complementos adicionales para el entorno de desarrollo.

A continuación, se añadió el repositorio oficial de EcEmma mediante la opción **Add**, indicando la siguiente dirección: <https://update.eclemma.org/>. De esta forma Eclipse puede acceder al repositorio oficial del plugin y descargar los componentes necesarios para su instalación.



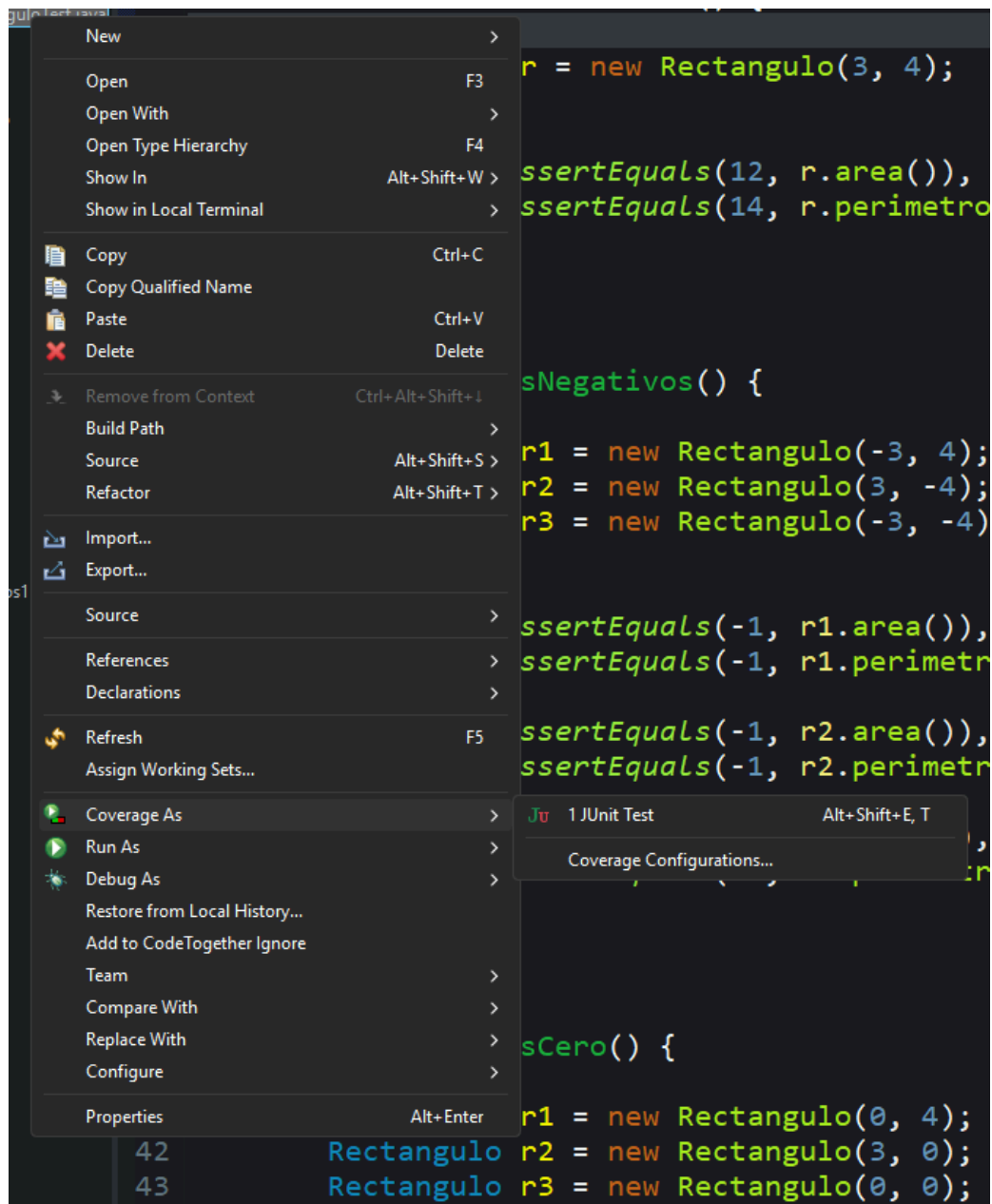


Una vez añadido el repositorio, Eclipse mostró los componentes disponibles para su instalación. Se seleccionó la opción **EclEmma Java Code Coverage**, que proporciona las herramientas necesarias para analizar la cobertura de código de los tests unitarios.



Tras completar el asistente de instalación, Eclipse descargó los componentes necesarios y solicitó reiniciar el IDE para finalizar correctamente la integración del plugin.

Una vez reiniciado Eclipse, quedó disponible la opción Coverage As, que permite ejecutar las pruebas unitarias obteniendo información detallada sobre la cobertura del código.

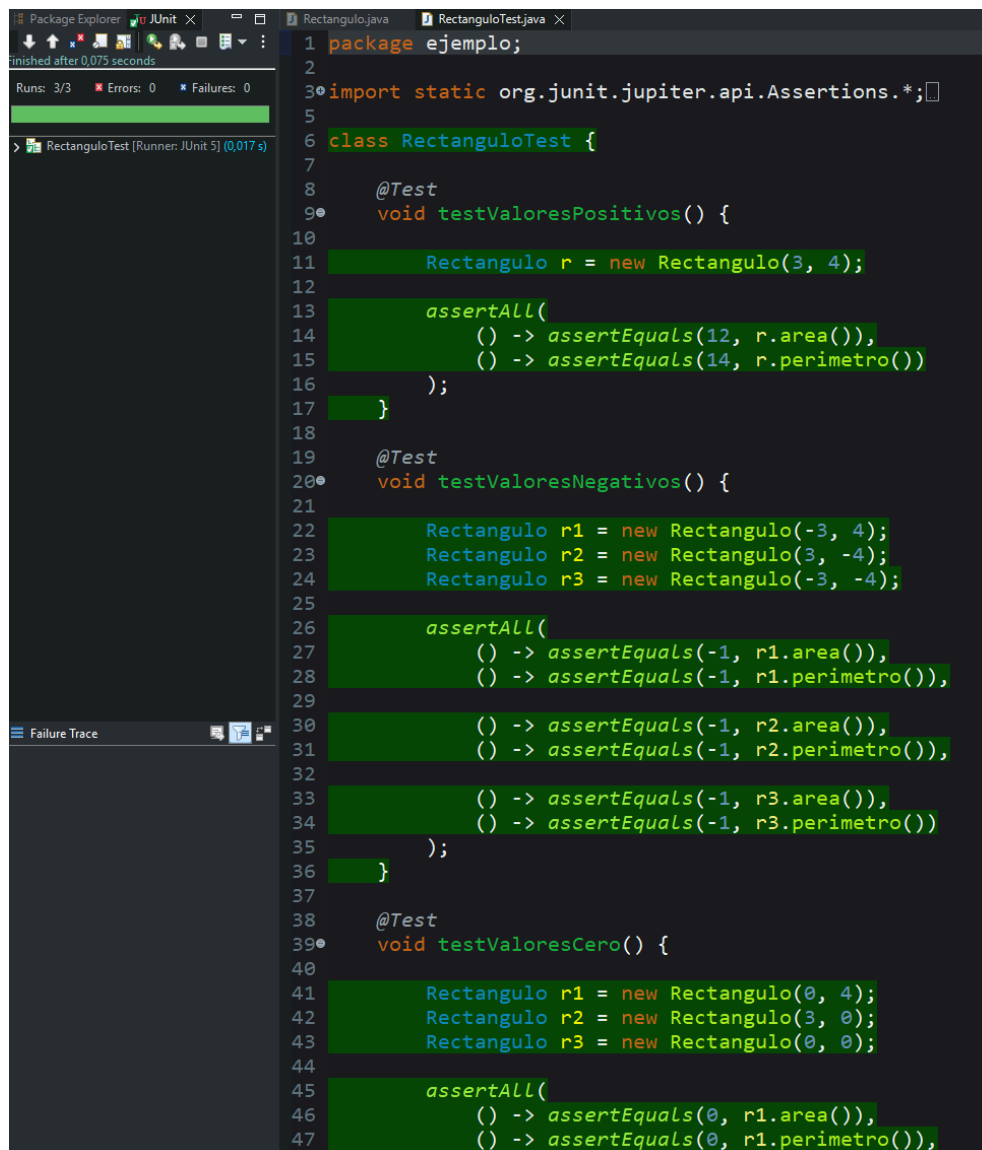


Ejecución de la cobertura de pruebas

Una vez instalado EcEmma, se utilizó el proyecto desarrollado en la práctica anterior para comprobar la cobertura de las pruebas unitarias realizadas sobre la clase Rectangulo.

Para ello se seleccionó la clase de pruebas y se ejecutó la opción Coverage As → JUnit Test. De esta forma, además de ejecutar los tests, EcEmma analiza qué partes del código han sido recorridas durante la ejecución.

En la siguiente imagen puede observarse la cobertura obtenida tras ejecutar las pruebas unitarias.



The screenshot shows an IDE with two tabs: Rectangulo.java and RectanguloTest.java. The left sidebar shows the Package Explorer with a tree view containing RectanguloTest [Runner: JUnit 5] (0.017 s). The main editor displays the RectanguloTest.java file with the following code:

```
1 package ejemplo;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5 class RectanguloTest {
6
7     @Test
8     void testValoresPositivos() {
9
10         Rectangulo r = new Rectangulo(3, 4);
11
12         assertEquals(12, r.area());
13         assertEquals(14, r.perimetro());
14     }
15
16     @Test
17     void testValoresNegativos() {
18
19         Rectangulo r1 = new Rectangulo(-3, 4);
20         Rectangulo r2 = new Rectangulo(3, -4);
21         Rectangulo r3 = new Rectangulo(-3, -4);
22
23         assertEquals(-1, r1.area());
24         assertEquals(-1, r1.perimetro());
25
26         assertEquals(-1, r2.area());
27         assertEquals(-1, r2.perimetro());
28
29         assertEquals(-1, r3.area());
30         assertEquals(-1, r3.perimetro());
31     }
32
33     @Test
34     void testValoresCero() {
35
36         Rectangulo r1 = new Rectangulo(0, 4);
37         Rectangulo r2 = new Rectangulo(3, 0);
38         Rectangulo r3 = new Rectangulo(0, 0);
39
40         assertEquals(0, r1.area());
41         assertEquals(0, r1.perimetro());
42     }
43 }
```

The code is annotated with JUnit 5 annotations and assertions. The left sidebar shows the test results: Runs: 3/3, Errors: 0, Failures: 0. The bottom of the sidebar shows a Failure Trace section.

```

1 package ejemplo;
2
3 public class Rectangulo {
4
5     private int ancho;
6     private int alto;
7
8     public Rectangulo(int ancho, int alto) {
9         this.ancho = ancho;
10        this.alto = alto;
11    }
12
13    public int area() {
14
15        if (ancho < 0 || alto < 0) {
16            return -1;
17        }
18
19        if (ancho == 0 || alto == 0) {
20            return 0;
21        }
22
23        return ancho * alto;
24    }
25
26    public int perimetro() {
27
28        if (ancho < 0 || alto < 0) {
29            return -1;
30        }
31
32        if (ancho == 0 || alto == 0) {
33            return 0;
34        }
35
36        return 2 * ancho + 2 * alto;
37    }
38 }

```

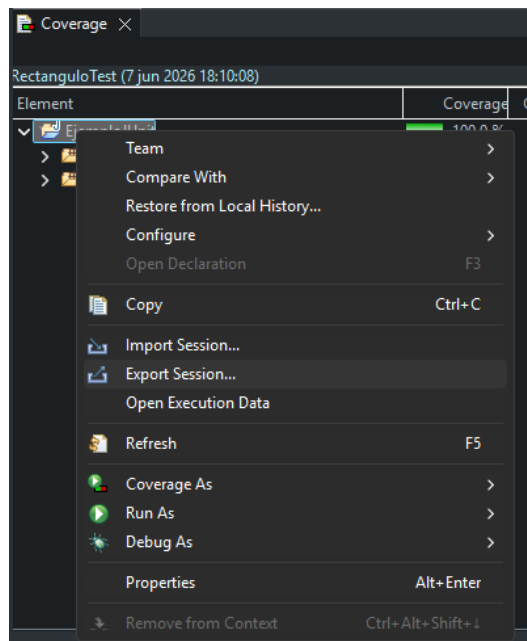
Resultados obtenidos

Tras ejecutar la cobertura se comprobó que todas las pruebas unitarias finalizaron correctamente. Los casos de prueba desarrollados para valores positivos, negativos y cero permitieron recorrer todos los caminos de ejecución de los métodos implementados.

Al analizar la clase `Rectangulo`, todas las instrucciones aparecieron marcadas en color verde, indicando que fueron ejecutadas durante las pruebas. Esto supone una cobertura completa del código, por lo que no fue necesario añadir nuevos juegos de datos ni crear pruebas adicionales.

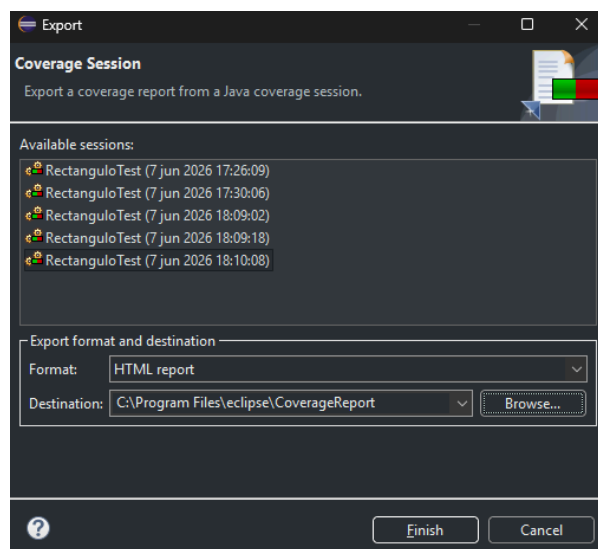
En conclusión, el análisis realizado con EcIEmma ha permitido verificar que las pruebas unitarias desarrolladas cubren completamente el comportamiento de la clase `Rectangulo`, garantizando que todas las ramas de ejecución han sido comprobadas correctamente.

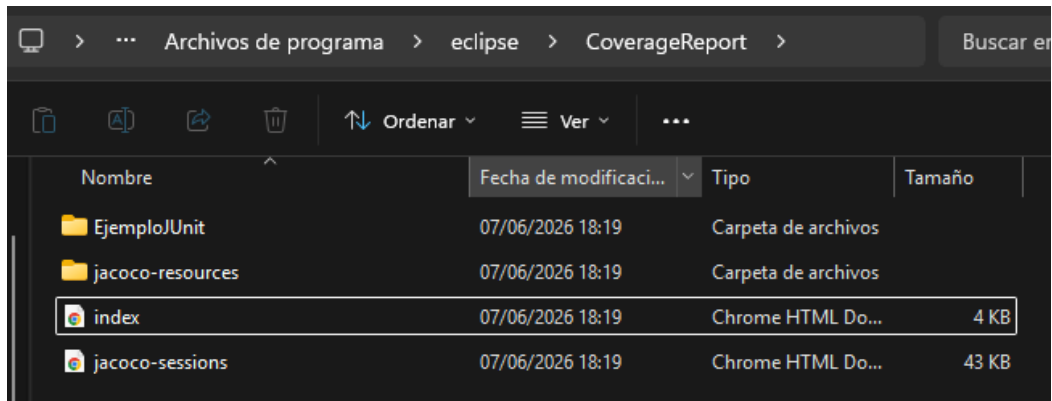
Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ EjemploJUnit	100,0 %	254	0	254
▼ src	100,0 %	57	0	57
▼ ejemplo	100,0 %	57	0	57
> Rectangulo.java	100,0 %	57	0	57
▼ test	100,0 %	197	0	197
▼ ejemplo	100,0 %	197	0	197
> RectanguloTest.java	100,0 %	197	0	197



Generación del informe de cobertura

Una vez finalizado el análisis de cobertura, se utilizó la opción **Export Session** para generar un informe detallado de los resultados obtenidos. Para ello se seleccionó el formato **HTML report** y se indicó una carpeta de destino donde almacenar los archivos generados por EcEmma.





El informe HTML generado confirma una cobertura del 100% tanto en instrucciones como en ramas de ejecución, sin elementos pendientes de ser evaluados por las pruebas.

RectanguloTest (7 jun 2026 18:10:08)

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Ctxy	Missed	Lines	Missed	Methods	Missed	Classes
EjemploJUnit	0 of 254	100 %	0 of 16	100 %	0	29	0	42	0	21	0	2
Total	0 of 254	100 %	0 of 16	100 %	0	29	0	42	0	21	0	2

RectanguloTest (7 jun 2026 18:10:08)

Created with JaCoCo 0.8.14.202510111229

Mediante el uso de EclEmma se ha comprobado que las pruebas unitarias desarrolladas para la clase **Rectangulo** cubren completamente todas las instrucciones y ramas de ejecución del código. Además, se ha generado un informe HTML que permite consultar de forma detallada los resultados obtenidos durante el análisis de cobertura. Por tanto, puede concluirse que el conjunto de pruebas desarrollado ofrece una validación completa del comportamiento implementado.